

error on our training set is very low and the error on our test set is very high.

We can define training and test sets as follows:

```
import numpy as np
np.random.seed(518)
ess_shuffled = ess.reindex(np.random.permutation(ess.index)).reset_index(drop =
True)
training_data = ess_shuffled.loc[0:37000,:]
test_data = ess_shuffled.loc[37001:,:].reset_index(drop = True)
```

In this snippet, we used the `numpy` module to shuffle the data—in other words, keeping all the data but moving the rows randomly. We accomplished this with the `reindex()` method of the `pandas` module. The reindexing is done with a random shuffling of the row numbers, which we get by using the `numpy` module’s permutation capability. After shuffling the dataset, we select the first 37,000 shuffled rows as a training dataset, and the remainder of the rows as a test dataset. The command `np.random.seed(518)` is not necessary, but if you run it you’ll ensure that you’ll get the same pseudo-random results that we show here.

After defining our training and test data, we generate a decision tree using only the training data:

```
thetree = getsplit(0,training_data,variables,outcome_variable)
```

Finally, we check the average error rate on the test data, which wasn’t used to train our decision tree:

```
predictions = []
for k in range(0,len(test_data.index)):
    observation = test_data.loc[k,:]
    predictions.append(get_prediction(observation,thetree))

test_data.loc[:,'predicted'] = predictions
errors = abs(test_data.loc[:,'predicted'] - test_data.loc[:,'happy'])
print(np.mean(errors))
```

We find that our mean error rate on the test data is 1.371. This is just a hair higher than the 1.369 error rate we found when we used the whole dataset for both training and testing. This indicates that our model doesn’t suffer